

GS2026.1: Pensamento Computacional e Automação com Python

Mission Control AI: Sistema Inteligente de Monitoramento de Missão Espacial

1. Contexto

A indústria espacial depende de sistemas capazes de acompanhar o funcionamento de missões, satélites, sondas e módulos experimentais.

Durante uma missão espacial, diferentes informações precisam ser analisadas continuamente, como temperatura, comunicação, bateria, oxigênio e estabilidade operacional. A partir desses dados, um sistema computacional pode identificar riscos, emitir alertas e sugerir ações para apoiar a tomada de decisão.

Nesta Global Solution, sua equipe deverá desenvolver um sistema em Python chamado **Mission Control AI**, capaz de simular o monitoramento inteligente de uma missão espacial experimental.

O sistema deverá organizar dados simulados da missão, analisar esses dados automaticamente e apresentar um relatório final com a situação da operação.

2. Objetivo da atividade

Desenvolver um programa em Python que simule um sistema básico de controle de missão espacial.

O programa deverá:

- armazenar dados simulados de uma missão;
- analisar diferentes ciclos de monitoramento;
- gerar alertas automáticos;
- calcular o nível de risco de cada ciclo;
- identificar a tendência da missão;
- indicar qual área da missão foi mais afetada;
- exibir um relatório final no terminal;
- documentar o projeto em um repositório no GitHub.

3. Desafio

Sua equipe foi contratada para criar a primeira versão do **Mission Control AI**, um sistema que auxilia o acompanhamento de uma missão espacial experimental.

A missão será simulada por meio de **ciclos de monitoramento**.

Cada ciclo representa um momento da missão.

Exemplo:

Ciclo 1 – início da missão
 Ciclo 2 – estabilização dos sistemas
 Ciclo 3 – queda parcial de comunicação
 Ciclo 4 – alerta de energia
 Ciclo 5 – risco operacional
 Ciclo 6 – tentativa de recuperação

Em cada ciclo, o sistema deverá analisar os dados da missão e responder automaticamente:

A missão está estável?
 A missão está em atenção?
 A missão está crítica?
 Qual área da missão apresenta maior risco?
 A situação geral está melhorando ou piorando?
 Qual recomendação deve ser apresentada?

4. Estrutura principal dos dados

O sistema deverá ter uma matriz principal chamada `dados_missao`.

Essa matriz será a base do projeto.

4.1. O que é a matriz `dados_missao`?

A matriz `dados_missao` é uma lista de listas.

Cada **linha** representa um ciclo da missão.

Cada **coluna** representa uma informação monitorada.

A estrutura deve seguir este modelo:

```
dados_missao = [
    [temperatura, comunicacao, bateria, oxigenio, estabilidade],
    [temperatura, comunicacao, bateria, oxigenio, estabilidade],
    [temperatura, comunicacao, bateria, oxigenio, estabilidade],
    [temperatura, comunicacao, bateria, oxigenio, estabilidade],
    [temperatura, comunicacao, bateria, oxigenio, estabilidade],
    [temperatura, comunicacao, bateria, oxigenio, estabilidade]
]
```

A equipe deve criar no mínimo **6 ciclos de monitoramento**.

4.2. Significado de cada coluna

Cada linha da matriz deve seguir exatamente esta ordem:

```
[temperatura, comunicacao, bateria, oxigenio, estabilidade]
```

Posição	Informação	Exemplo	Significado
0	Temperatura	24	Temperatura interna do módulo, em °C
1	Comunicação	92	Qualidade do sinal de comunicação, em %
2	Bateria	88	Nível de bateria da missão, em %
3	Oxigênio	96	Nível de oxigênio disponível, em %
4	Estabilidade	90	Estabilidade geral dos sistemas, em %

4.3. Exemplo de matriz com dados simulados

A equipe pode criar seus próprios dados. Abaixo há um exemplo:

```
dados_missao = [
    [24, 92, 88, 96, 90],
    [27, 80, 72, 94, 85],
    [31, 65, 58, 91, 70],
    [36, 42, 38, 87, 55],
    [39, 28, 19, 78, 35],
    [34, 55, 32, 82, 50]
]
```

4.4. Como interpretar uma linha da matriz

A linha abaixo:

```
[24, 92, 88, 96, 90]
```

representa um ciclo da missão com os seguintes dados:

```
Temperatura: 24 °C
Comunicação: 92%
Bateria: 88%
Oxigênio: 96%
Estabilidade: 90%
```

Essa situação indica um ciclo provavelmente estável.

Agora observe esta linha:

```
[39, 28, 19, 78, 35]
```

Ela representa:

```
Temperatura: 39 °C
Comunicação: 28%
Bateria: 19%
Oxigênio: 78%
Estabilidade: 35%
```

Essa situação indica um ciclo provavelmente crítico, pois há temperatura alta, comunicação baixa, bateria baixa, oxigênio baixo e baixa estabilidade.

5. Áreas monitoradas da missão

O sistema deverá ter uma lista com as áreas monitoradas. Exemplo:

```
areas_monitoradas = [
    "Temperatura interna",
    "Comunicação com a base",
    "Sistema de energia",
    "Suporte de oxigênio",
    "Estabilidade operacional"
]
```

Essa lista deve estar relacionada às colunas da matriz dados_missao.

temperatura → Temperatura interna
 comunicacao → Comunicação com a base
 bateria → Sistema de energia
 oxigenio → Suporte de oxigênio
 estabilidade → Estabilidade operacional

O sistema deverá usar essas áreas para exibir o relatório final e identificar qual delas acumulou maior risco durante a missão.

6. Regras de alerta

O sistema deverá classificar cada informação analisada como:

NORMAL
 ATENÇÃO
 CRÍTICO

A equipe pode ajustar os limites, desde que explique as regras no README .md.

Abaixo está uma sugestão de regras.

6.1. Temperatura

Condição	Classificação
menor que 18 °C	ATENÇÃO
de 18 °C até 30 °C	NORMAL
maior que 30 °C até 35 °C	ATENÇÃO
maior que 35 °C	CRÍTICO

6.2. Comunicação

Condição	Classificação
menor que 30%	CRÍTICO
de 30% até 59%	ATENÇÃO
60% ou mais	NORMAL

6.3. Bateria

Condição	Classificação
menor que 20%	CRÍTICO
de 20% até 49%	ATENÇÃO
50% ou mais	NORMAL

6.4. Oxigênio

Condição	Classificação
menor que 80%	CRÍTICO
de 80% até 89%	ATENÇÃO
90% ou mais	NORMAL

6.5. Estabilidade

Condição	Classificação
menor que 40%	CRÍTICO
de 40% até 69%	ATENÇÃO
70% ou mais	NORMAL

7. Pontuação de risco

Cada classificação deve gerar uma pontuação.

NORMAL = 0 ponto
 ATENÇÃO = 1 ponto
 CRÍTICO = 2 pontos

Como cada ciclo possui 5 informações monitoradas, a pontuação máxima de um ciclo será 10 pontos.

Exemplo:

Temperatura: CRÍTICO = 2 pontos
 Comunicação: ATENÇÃO = 1 ponto
 Bateria: CRÍTICO = 2 pontos
 Oxigênio: NORMAL = 0 ponto
 Estabilidade: ATENÇÃO = 1 ponto

 Total do ciclo: 6 pontos

8. Classificação do ciclo

Depois de calcular a pontuação total, o sistema deverá classificar cada ciclo.

Pontuação total	Classificação
0 a 2 pontos	MISSÃO ESTÁVEL
3 a 5 pontos	MISSÃO EM ATENÇÃO
6 a 10 pontos	MISSÃO CRÍTICA

9. Análise da tendência da missão

O sistema deverá comparar o risco do primeiro ciclo com o risco do último ciclo.

A partir dessa comparação, o programa deverá informar se a missão:

- melhorou;
- piorou;
- permaneceu estável.

Regras sugeridas:

Se o risco do último ciclo for maior que o risco do primeiro ciclo:
 A missão apresentou tendência de piora.

Se o risco do último ciclo for menor que o risco do primeiro ciclo:
 A missão apresentou tendência de melhora.

Se o risco do último ciclo for igual ao risco do primeiro ciclo:
 A missão permaneceu estável em relação ao início.

10. Identificação da área mais afetada

O sistema deverá somar a pontuação de risco de cada área ao longo de todos os ciclos.

Exemplo:

Temperatura interna: 4 pontos Comunicação com a base: 5 pontos Sistema de energia: 6 pontos Suporte de oxigênio: 3 pontos Estabilidade operacional: 4 pontos

Nesse caso, a área mais afetada seria:

Sistema de energia

Essa informação deverá aparecer no relatório final.

11. Recomendações automáticas

O sistema deverá gerar recomendações automáticas com base nos dados analisados.

Exemplos:

Temperatura crítica: verificar controle térmico da missão. Comunicação crítica: tentar restabelecer contato com a base. Bateria crítica: ativar modo de economia de energia. Oxigênio crítico: acionar protocolo de suporte à vida. Estabilidade crítica: reduzir operações não essenciais.
--

A recomendação pode aparecer em cada ciclo ou apenas no relatório final.

12. Exemplo de como o sistema deve aparecer no terminal

A saída não precisa ser idêntica ao exemplo abaixo, mas o sistema deve apresentar informações equivalentes.

Considerando a seguinte matriz:

```
dados_missao = [  
    [24, 92, 88, 96, 90],  
    [27, 80, 72, 94, 85],  
    [31, 65, 58, 91, 70],  
    [36, 42, 38, 87, 55],  
    [39, 28, 19, 78, 35],  
    [34, 55, 32, 82, 50]  
]
```

A saída no terminal poderia ser:

```
=====
MISSION CONTROL AI
=====
Missão: Orion Test Alpha
Equipe: Equipe Apollo
Quantidade de ciclos analisados: 6
=====

CICLO 1
-----
Temperatura: 24 °C | NORMAL | Temperatura estável
Comunicação: 92% | NORMAL | Comunicação estável
Bateria: 88% | NORMAL | Energia estável
Oxigênio: 96% | NORMAL | Oxigênio adequado
Estabilidade: 90% | NORMAL | Estabilidade operacional adequada

Pontuação de risco do ciclo: 0
Classificação do ciclo: MISSÃO ESTÁVEL
Recomendação: Manter operação normal e continuar monitoramento.

CICLO 2
-----
Temperatura: 27 °C | NORMAL | Temperatura estável
Comunicação: 80% | NORMAL | Comunicação estável
Bateria: 72% | NORMAL | Energia estável
Oxigênio: 94% | NORMAL | Oxigênio adequado
Estabilidade: 85% | NORMAL | Estabilidade operacional adequada

Pontuação de risco do ciclo: 0
Classificação do ciclo: MISSÃO ESTÁVEL
```

Recomendação: Manter operação normal e continuar monitoramento.

CICLO 3

 Temperatura: 31 °C | ATENÇÃO | Temperatura elevada
 Comunicação: 65% | NORMAL | Comunicação estável
 Bateria: 58% | NORMAL | Energia estável
 Oxigênio: 91% | NORMAL | Oxigênio adequado
 Estabilidade: 70% | NORMAL | Estabilidade operacional adequada

Pontuação de risco do ciclo: 1
 Classificação do ciclo: MISSÃO ESTÁVEL
 Recomendação: Verificar controle térmico da missão.

CICLO 4

 Temperatura: 36 °C | CRÍTICO | Risco de superaquecimento
 Comunicação: 42% | ATENÇÃO | Comunicação instável
 Bateria: 38% | ATENÇÃO | Bateria abaixo do recomendado
 Oxigênio: 87% | ATENÇÃO | Oxigênio abaixo do ideal
 Estabilidade: 55% | ATENÇÃO | Estabilidade operacional reduzida

Pontuação de risco do ciclo: 6
 Classificação do ciclo: MISSÃO CRÍTICA
 Recomendação: Verificar controle térmico da missão.

CICLO 5

 Temperatura: 39 °C | CRÍTICO | Risco de superaquecimento
 Comunicação: 28% | CRÍTICO | Comunicação com a base em nível crítico
 Bateria: 19% | CRÍTICO | Bateria em nível crítico
 Oxigênio: 78% | CRÍTICO | Oxigênio em nível crítico
 Estabilidade: 35% | CRÍTICO | Estabilidade operacional crítica

Pontuação de risco do ciclo: 10
 Classificação do ciclo: MISSÃO CRÍTICA
 Recomendação: Ativar modo de segurança e priorizar suporte à vida, energia e comunicação.

CICLO 6

 Temperatura: 34 °C | ATENÇÃO | Temperatura elevada
 Comunicação: 55% | ATENÇÃO | Comunicação instável
 Bateria: 32% | ATENÇÃO | Bateria abaixo do recomendado
 Oxigênio: 82% | ATENÇÃO | Oxigênio abaixo do ideal
 Estabilidade: 50% | ATENÇÃO | Estabilidade operacional reduzida

Pontuação de risco do ciclo: 5
 Classificação do ciclo: MISSÃO EM ATENÇÃO
 Recomendação: Monitorar sistemas em atenção e preparar plano de

contingência.

=====

RELATÓRIO FINAL DA MISSÃO

=====

Missão: Orion Test Alpha

Equipe: Equipe Apollo

Quantidade de ciclos analisados: 6

Média de temperatura: 31.83 °C

Média de comunicação: 60.33%

Média de bateria: 51.17%

Média de oxigênio: 88.00%

Média de estabilidade: 64.17%

Ciclo mais crítico: Ciclo 5

Maior pontuação de risco: 10

Risco médio da missão: 3.67

Quantidade de ciclos críticos: 2

Tendência da missão:

A missão apresentou tendência de piora.

Pontuação acumulada por área:

Temperatura interna: 5 pontos

Comunicação com a base: 3 pontos

Sistema de energia: 3 pontos

Suporte de oxigênio: 3 pontos

Estabilidade operacional: 3 pontos

Área mais afetada:

Temperatura interna

Classificação final da missão:

MISSÃO EM ATENÇÃO

Conclusão:

A missão apresentou instabilidade relevante durante a operação. Apesar da tentativa de recuperação no último ciclo, ainda existem sistemas em atenção e a equipe deve manter o plano de contingência ativo.

13. Requisitos obrigatórios do código

O código Python deverá conter:

1. Nome da missão.
2. Nome da equipe.
3. Matriz dados_missão com pelo menos 6 ciclos.
4. Cada ciclo com 5 informações nesta ordem:
 - a. temperatura;
 - b. comunicação;
 - c. bateria;
 - d. oxigênio;
 - e. estabilidade.
5. Lista com as áreas monitoradas.
6. Pelo menos 5 funções.
7. Estrutura de repetição para percorrer os ciclos.
8. Estruturas condicionais para gerar alertas.
9. Cálculo de risco por ciclo.
10. Classificação de cada ciclo.
11. Análise da tendência da missão.
12. Identificação da área mais afetada.
13. Relatório final exibido no terminal.

14. Funções sugeridas

O projeto deve ter pelo menos **5 funções**.

Sugestão de funções:

```
analisar_temperatura()
analisar_comunicacao()
analisar_bateria()
analisar_oxigenio()
analisar_estabilidade()
classificar_ciclo()
analisar_tendencia()
identificar_area_mais_afetada()
gerar_recomendacao()
gerar_relatorio_final()
```

A equipe pode criar outras funções, desde que o projeto cumpra todos os requisitos obrigatórios.

15. Entrega no GitHub

A entrega deverá ser feita por meio de um repositório no GitHub.

O repositório deve conter:

```
mission-control-ai/
|
├── README.md
└── mission_control.py
```

16. Entrega no portal da FIAP

No portal da FIAP (em Entregas de Trabalho), a equipe deverá entregar apenas um arquivo .txt.

Esse arquivo deve conter:

Nome da missão:
 Nome da equipe:
 Integrantes:
 RM dos integrantes:
 Link do repositório GitHub:
 Link do vídeo pitch no YouTube:

Exemplo:

Nome da missão: Orion Test Alpha
 Nome da equipe: Equipe Apollo

Integrantes:
 Maria Silva - RM:000001
 João Santos - RM: 000002
 Pedro Lima - RM: 000003

Link do repositório GitHub:
<https://github.com/usuario/mission-control-ai>

Link do vídeo pitch no YouTube:
<https://youtube.com/...>

18. Critérios de avaliação

A atividade vale **10 pontos**.

- **Funcionamento geral do sistema (2,0 pontos)**
 - programa executa corretamente, analisa os ciclos e gera relatório final
- **Uso de matriz, listas e repetição (1,0 pontos)**
 - usa corretamente a matriz dados_missão com pelo menos 6 ciclos
- **Uso de funções (1,0 pontos)**
 - possui pelo menos 5 funções bem definidas e utilizadas
- **Lógica de decisão, risco e análise dos dados (1,0 pontos)**
 - gera alertas corretamente
 - calcula o risco de cada ciclo
 - identifica a tendência da missão
 - identifica a área mais afetada
- **GitHub, README e organização (1,0 pontos)**
 - repositório GitHub acessível
 - README .md completo
- **Vídeo Pitch (2,0 pontos)**
 - Vídeo pitch deve ser entregue nesta entrega de python E na entrega “GS2026.1 - VÍDEO PITCH”
 - Deve ser entregue o link do YouTube como não listado
 - Vídeo deve ter no máximo 3 minutos
 - Não é necessário que todos os integrantes apareçam no vídeo
 - Não é permitido narração com IA
 - Vídeo deve deixar claro a proposta da solução com tema da GS

19. Regras importantes

- O trabalho pode ser feito individualmente, em dupla ou em trio.
- O código deve executar sem erros.
- O código deve estar organizado e legível.
- A equipe pode usar dados simulados fixos no código.
- Não é obrigatório usar interface gráfica.
- Não é obrigatório usar bibliotecas externas.
- Não é obrigatório usar machine learning.
- A inteligência do sistema será baseada em regras lógicas.
- Cada equipe deve personalizar o nome da missão, os dados simulados e as informações do repositório.
- Códigos idênticos entre equipes diferentes poderão ter desconto na nota.

- O link do GitHub deve estar acessível no momento da correção.